GoFood

Project Overview — A MERN Stack Food Delivery Web Application

Tech Stack

Layer	Technology
Frontend	React 19, React Router, Bootstrap 5
Backend	Node.js, Express 5
Database	MongoDB Atlas (Mongoose)
Authentication	JWT + bcrypt
State Management	React Context + localStorage

Project Structure

```
GO FOOD/
├── backend/                  # Express API server (port 5000)
│   ├── index.js              # Server entry point
│   ├── db.js                 # MongoDB connection
│   ├── Routes/
│   │   ├── CreateUser.js     # Signup & login
│   │   ├── DisplayData.js    # Food menu data
│   │   └── OrderData.js      # Place & fetch orders
│   ├── models/
│   │   ├── User.js
│   │   └── Orders.js
│   ├── middleware/
│   │   └── fetchData.js      # JWT auth middleware
│   └── foodData2.json        # Seed data for import
│
└── my-app/                   # React frontend (port 3000)
    └── src/
        ├── App.js            # Routes
        ├── screens/          # Pages
        ├── components/       # Navbar, Card, Carousel, Footer
        └── context/          # Auth & Cart state
```

Pages & Routes

Route	Screen	Purpose
/	Home	Hero carousel, search, food menu
/login	Login	User login
/signup	Signup	User registration
/cart	Cart	View cart, checkout
/myorders	My Orders	Order history (logged-in users)

User Flow

1. **Browse** — User lands on Home, food menu loads from `/api/foodData`
2. **Filter** — User filters by category or searches by name/description
3. **Cart** — User adds items with size/qty; cart persists in localStorage
4. **Auth** — User signs up or logs in; JWT stored in localStorage
5. **Checkout** — User places order via `/api/orderData` (requires login)
6. **History** — User views past orders on My Orders page

API Endpoints

Method	Endpoint	Auth	Description
POST	/api/createuser	No	Register new user
POST	/api/loginuser	No	Login, returns JWT
POST	/api/foodData	No	Get food items + categories
POST	/api/orderData	Yes	Place an order
POST	/api/myorders	Yes	Get user's order history

Database Collections

Collection	Contents
users	name, email, password (hashed), location
food_items	name, img, description, options, CategoryName
foodCategory	CategoryName
orders	userId, order_data[], totalAmount, status, orderDate

Key Features Implemented

- Sticky navbar with cart badge count
- Hero carousel with search bar
- Category filter buttons + text search
- Food cards with size options and add-to-cart
- Persistent cart (localStorage)
- JWT-based authentication
- Order placement and order history
- Responsive UI (Bootstrap + custom CSS)

How to Run

Backend

```
cd backend
npm install
npm run dev          # Runs on http://localhost:5000
```

Frontend

```
cd my-app
npm install
npm start            # Runs on http://localhost:3000
```

Note: Backend requires a .env file with MONGO_URI and optionally JWT_SECRET.

Current Status

Completed

- User authentication (signup/login)
- Food menu display & search
- Shopping cart & checkout
- Order history
- Responsive UI

Not Yet Implemented

- Payment gateway integration
- Admin panel
- Order status updates (tracking)
- Email notifications
- Production deployment
